Welcome to Colab!

Google Colab is available in VS Code!

Try the new Google Colab extension for Visual Studio Code. You can get up and running in just a few clicks:

- In VS Code, open the ***Extensions*** view and search for 'Google Colab' to install.
- Open the kernel selector by creating or opening any `.ipynb` notebook file in your local workspace and either running a cell or clicking the ***Select kernel*** button in the top right.
- Click ***Colab*** and then select your desired runtime, sign in with your Google Account and you're all set!

See more details in our announcement blog here.

🎁 Free-of-charge Pro plan for Gemini and Colab for US university students 🎓

Get more access to our most accurate model Gemini 3 Pro for advanced coding, complex research and innovative projects, backed by Colab's dedicated high-compute resources for data science and machine learning.

Get the Gemini free-of-charge offer at gemini.google/students.
Get the Colab free-of-charge offer at colab.research.google.com/signup.
Terms apply.

Access popular AI models via Google Colab-AI without an API key

All users have access to most popular LLMs via the `google-colab-ai` Python library, and paid users have access to a wider selection of models. For more details, refer to getting started with Google Colab AI.

```
from google.colab import ai
response = ai.generate_text("What is the capital of France?")
```

Explore the Gemini API

The Gemini API gives you access to Gemini models created by Google DeepMind. Gemini models are built from the ground up to be multimodal, so you can reason seamlessly across text, images, code and audio.

How to get started

- Go to Google AI Studio and log in with your Google Account.
- Create an API key.
- Use a quickstart for Python or call the REST API using curl.

Discover Gemini's advanced capabilities

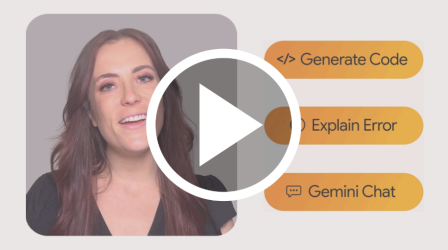
- Play with Gemini multimodal outputs, mixing text and images in an iterative way.
- Discover the multimodal Live API (demo here).
- Learn how to analyse images and detect items in your pictures using Gemini (bonus, there's a 3D version as well!).
- Unlock the power of the Gemini thinking model, capable of solving complex tasks with its inner thoughts.

Explore complex use cases

- Use Gemini grounding capabilities to create a report on a company based on what the model can find on the Internet.
- Extract invoices and form data from PDFs in a structured way.
- Create illustrations based on a whole book using Gemini large context window and Imagen.

To learn more, take a look at the Gemini cookbook or visit the Gemini API documentation.

Colab now has AI features powered by Gemini. The video below provides information on how to use these features, whether you're new to Python or a seasoned veteran.



What is Colab?

Colab, or 'Colaboratory', allows you to write and execute Python in your browser, with

- Zero configuration required
- Access to GPUs free of charge
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch Introduction to Colab or Colab features you may have missed to learn more or just get started below!

Getting started

The document that you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable and prints the result:

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

```
86400
```

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut 'Command/Ctrl+Enter'. To edit the code, just click the cell and start editing.

Variables that you define in one cell can later be used in other cells:

```
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week
```

```
604800
```

Colab notebooks allow you to combine **executable code** and **rich text** in a single document, along with **images**, **HTML**, **LaTeX** and more. When you create your own Colab notebooks, they are stored in your Google Drive account. You can easily share your Colab notebooks with co-workers or friends, allowing them to comment on your notebooks or even edit them. To find out more, see Overview of Colab. To create a new Colab notebook you can use the File menu above, or use the following link: Create a new Colab notebook.

Colab notebooks are Jupyter notebooks that are hosted by Colab. To find out more about the Jupyter project, see jupyter.org.

Data science

With Colab you can harness the full power of popular Python libraries to analyse and visualise data. The code cell below uses **numpy** to generate some random data, and uses **matplotlib** to visualise it. To edit the code, just click the cell and start editing.

You can import your own data into Colab notebooks from your Google Drive account, including from spreadsheets, as well as from GitHub and many other sources. To find out more about importing data, and how Colab can be used for data science, see the links below under Working with data.

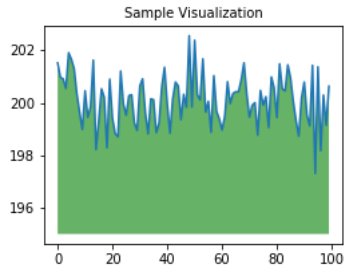
```
import numpy as np
import IPython.display as display
from matplotlib import pyplot as plt
import io
import base64

ys = 200 + np.random.randn(100)
x = [x for x in range(len(ys))]

fig = plt.figure(figsize=(4, 3), facecolor='w')
```

```
plt.plot(x, ys, '-')
plt.fill_between(x, ys, 195, where=(ys > 195), facecolor='g', alpha=0.6)
plt.title("Sample Visualization", fontsize=10)

data = io.BytesIO()
plt.savefig(data)
image = F"{base64.b64encode(data.getvalue()).decode()}"
alt = "Sample Visualization"
display.display(display.Markdown(F"""![{alt}]({image})"""))
plt.close(fig)
```



Colab notebooks execute code on Google's cloud servers, meaning that you can leverage the power of Google hardware, including GPUs and TPUs, regardless of the power of your machine. All you need is a browser.

For example, if you find yourself waiting for **pandas** code to finish running and want to go faster, you can switch to a GPU runtime and use libraries like RAPIDS cuDF that provide zero-code-change acceleration.

To learn more about accelerating pandas on Colab, see the 10-minute guide or US stock market data analysis demo.

Machine learning

With Colab you can import an image dataset, train an image classifier on it and evaluate the model, all in just a few lines of code.

Colab is used extensively in the machine learning community with applications including:

- Getting started with TensorFlow
- Developing and training neural networks
- Experimenting with TPUs
- Disseminating AI research
- Creating tutorials

To see sample Colab notebooks that demonstrate machine learning applications, see the machine learning examples below.

More resources

Working with notebooks in Colab

- Overview of Colab
- Guide to markdown
- Importing libraries and installing dependencies
- Saving and loading notebooks in GitHub
- Interactive forms
- Interactive widgets

Working with data

- Loading data: Drive, Sheets and Google Cloud Storage
- Charts: visualising data
- Getting started with BigQuery

Machine learning

These are a few of the notebooks related to machine learning, including Google's online machine learning course. See the full course website for more.

- Intro to Pandas DataFrame
- Intro to RAPIDS cuDF to accelerate pandas

- Getting started with cuML's accelerator mode

Using accelerated hardware

- Train a CNN to classify handwritten digits on the MNIST dataset using Flax NNX API
- Train a Vision Transformer (ViT) for image classification with JAX
- Text classification with a transformer language model using JAX

Featured examples

- Train a miniGPT language model with JAX AI stack
- LoRA/QLoRA finetuning for LLM using Tunix
- Parameter-efficient fine-tuning of Gemma with LoRA and QLoRA
- Loading Hugging Face transformers checkpoints
- 8-bit integer quantisation in Keras
- Float8 training and inference with a simple transformer model
- Pretraining a transformer from scratch with KerasHub
- Simple MNIST convnet
- Image classification from scratch using Keras 3
- Image classification with KerasHub

```
!pip install pandas
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.
```

```
import pandas as pd
import numpy as np
```

```
print(pd.__version__)
```

```
2.2.2
```

```
series
A one-dimensional labeled array capable of holding data of any type such as integers, strings, floats, or objects.

It contains:

Data values Index labels
```

```
s=pd.Series([1,2,3,4,5])
```

```
print(s)
```

```
0    1
1    2
2    3
3    4
4    5
dtype: int64
```

```
s=pd.Series([1,2,3,4,5],index=['a','b','c','d','e'])
```

```
student={
    "name":"harshi",
    "branch":"aiml"
}
se=pd.Series(student)
print(se)
```

```
name      harshi
branch      aiml
dtype: object
```

```
import pandas as pd
n=pd.Series([10,20,30,40,50])
n*2
```

	0
0	20
1	40
2	60
3	80
4	100

dtype: int64

```
s+10
```

	0
a	11
b	12
c	13
d	14
e	15

dtype: int64

```
n.mean()
```

```
np.float64(30.0)
```

```
n.min()
```

```
10
```

```
n.sum()
```

```
np.int64(150)
```

```
n.describe()
```

	0
count	5.000000
mean	30.000000
std	15.811388
min	10.000000
25%	20.000000
50%	30.000000
75%	40.000000
max	50.000000

dtype: float64

```
n[n>20]
```

	0
2	30
3	40
4	50

dtype: int64

```
s[s>3]
```

	0
d	4
e	5

dtype: int64

```
names=pd.Series(['Nandu','Harshi','Bharathi'])
names.str.lower()
```

	0
0	nandu
1	harshi
2	bharathi

dtype: object

```
names.str.len()
```

	0
0	5
1	6
2	8

dtype: int64

```
names.str.lower()
```

	0
0	nandu
1	harshi
2	bharathi

dtype: object

```
names.str.startswith('N')
```

	0
0	True
1	False
2	False

dtype: bool

```
students={
    "names":["nandu","harshi","bharathi"],
    "branch":["aiml","aiml","aiml"]
}
s=pd.DataFrame(students)
print(s)
```

```
     names branch
0    nandu   aiml
1   harshi   aiml
2 bharathi   aiml
```

```
std=[
    {"name":"nandu","branch":"aiml"},
    {"name":"harshi","branch":"aiml"},
    {"name":"bharathi","branch":"aiml"}
]
s=pd.DataFrame(std)
print(s)
```

```
      name branch
0    nandu   aiml
```

```
1    harshi   aiml
2  bharathi   aiml
```

```
arr=np.random.rand(3,4)
c=pd.DataFrame(arr,columns=['A','B','C','D'])
print(c)
```

```
          A         B         C         D
0  0.079988  0.774943  0.105501  0.341717
1  0.906883  0.928789  0.559911  0.289490
2  0.536193  0.530301  0.579949  0.960988
```

```
data={
    'name':['sravya','harshitha','mounisha','punitha'],
    'age':[19,20,21,20],
    'score':[96,75,83,91],
    'grade':['A','C','B','A']
}
d=pd.DataFrame(data)
print(d)
```

```
        name  age  score grade
0     sravya   19     96     A
1  harshitha   20     75     C
2   mounisha   21     83     B
3    punitha   20     91     A
```

```
data={
    'name':['sravya','harshitha','mounisha','punitha'],
    'age':[19,20,21,20],
    'score':[96,75,83,91],
    'grade':['A','C','B','A']
}
d=pd.DataFrame(data)
```

```
d.head(3)
```

	name	age	score	grade
0	sravya	19	96	A
1	harshitha	20	75	C
2	mounisha	21	83	B

```
d.tail(2)
```

	name	age	score	grade
2	mounisha	21	83	B
3	punitha	20	91	A

```
d.shape
```

```
(4, 4)
```

```
d.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype 
---  ------  --------------  ----- 
 0   name    4 non-null      object
 1   age     4 non-null      int64 
 2   score   4 non-null      int64 
 3   grade   4 non-null      object
dtypes: int64(2), object(2)
memory usage: 260.0+ bytes
```

```
d.describe()
```

	age	score
count	4.000000	4.000000
mean	20.000000	86.250000
std	0.816497	9.215024
min	19.000000	75.000000
25%	19.750000	81.000000
50%	20.000000	87.000000
75%	20.250000	92.250000
max	21.000000	96.000000

```
d.index
```

```
RangeIndex(start=0, stop=4, step=1)
```

```
d.dtypes
```

	0
name	object
age	int64
score	int64
grade	object

dtype: object

```
d.nunique()
```

	0
name	4
age	3
score	4
grade	3

dtype: int64

```
d.isnull().sum()
```

	0
name	0
age	0
score	0
grade	0

dtype: int64

```
d['name']
```

	name
0	sravya
1	harshitha
2	mounisha
3	punitha

dtype: object

```
d.name
```

	name
0	sravya
1	harshitha
2	mounisha
3	punitha

dtype: object

```
d[['name','age']]
```

	name	age
0	sravya	19
1	harshitha	20
2	mounisha	21
3	punitha	20

```
d.loc[0]
```

	0
name	sravya
age	19
score	96
grade	A

dtype: object

```
d.loc[0:2]
```

	name	age	score	grade
0	sravya	19	96	A
1	harshitha	20	75	C
2	mounisha	21	83	B

```
d.loc[0:2,'name':'score']
```

	name	age	score
0	sravya	19	96
1	harshitha	20	75
2	mounisha	21	83

```
d.iloc[0]
```

	0
name	sravya
age	19
score	96
grade	A

dtype: object

```
d.iloc[0:2]
```

	name	age	score	grade
0	sravya	19	96	A
1	harshitha	20	75	C

```
d.iloc[0:2,0:2]
```

	name	age
0	sravya	19
1	harshitha	20

```
d.iloc[-1]
```

	3
name	punitha
age	20
score	91
grade	A

dtype: object

```
d.loc[0,'name']
```

```
'sravya'
```

```
d.iloc[0,0]
```

```
'sravya'
```

```
d[(d['score'] > 80) & (d['grade'] == 'A')]
d[(d['age'] < 24) | (d['score'] > 90)]
```

	name	age	score	grade
0	sravya	19	96	A
1	harshitha	20	75	C
2	mounisha	21	83	B
3	punitha	20	91	A

```
d.query('score>80 and age<25')
d.query('grade=="A"')
```

	name	age	score	grade
0	sravya	19	96	A
3	punitha	20	91	A

```
d[d['name'].isin(['sravya','mounisha'])]
```

	name	age	score	grade
0	sravya	19	96	A
2	mounisha	21	83	B

```
d['example']=["a","b","c","d"]
print(d)
```

```
        name  age  score  grade example
0     sravya   19    106   True       a
1  harshitha   20     85  False       b
2   mounisha   21     93   True       c
3    punitha   20    101   True       d
```

```
d['grade']=d['score']>=80
```

```
print(d)
```

```
        name  age  score  grade example
0     sravya   19    106   True       a
1  harshitha   20     85   True       b
2   mounisha   21     93   True       c
3    punitha   20    101   True       d
```

```
d['score']=d['score']+5
```

```
print(d)
```

```
        name  age  score  grade example
0     sravya   19    111   True       a
1  harshitha   20     90   True       b
2   mounisha   21     98   True       c
3    punitha   20    106   True       d
```

```
def grade(score):
    if score >= 90:
        return 'A'
    elif score >= 80:
        return 'B'
    else:
        return 'C'


d['Grade_Letter'] = d['score'].apply(grade)

print(d)
```

```
        name  age  score  grade example Grade_Letter
0     sravya   19    111   True       a            A
1  harshitha   20     90   True       b            A
2   mounisha   21     98   True       c            A
3    punitha   20    106   True       d            A
```

```
result={
    True:"pass",
    False:"fail"
}
pas={'A':90,'B':80,'D':60}
d['pass']=d['grade'].map(pas)
d['result']=d['grade'].map(result)
print(d)
```

```
        name  age  score  grade example Grade_Letter  pass result
0     sravya   19    111   True       a            A   NaN   pass
1  harshitha   20     90   True       b            A   NaN   pass
2   mounisha   21     98   True       c            A   NaN   pass
3    punitha   20    106   True       d            A   NaN   pass
```

```
d.drop('example',axis=1,inplace=True)
```

```
print(d)
```

```
        name  age  score  grade Grade_Letter  pass result
0     sravya   19    111   True            A   NaN   pass
1  harshitha   20     90   True            A   NaN   pass
2   mounisha   21     98   True            A   NaN   pass
3    punitha   20    106   True            A   NaN   pass
```

```
d.rename(columns={'score':'marks','name':'student'},inplace=True)
```

```
print(d)
```

```
     student  age  marks  grade Grade_Letter  pass result
0     sravya   19    111   True            A   NaN   pass
1  harshitha   20     90   True            A   NaN   pass
2   mounisha   21     98   True            A   NaN   pass
3    punitha   20    106   True            A   NaN   pass
```

```
d.replace('A',np.nan,inplace=True)
```

```
/tmp/ipykernel_2028/1056631716.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a
  d.replace('A',np.nan,inplace=True)
```

```
print(d)
```

```
     student  age  marks  grade  Grade_Letter  pass result
0     sravya   19    111   True           NaN   NaN   pass
1  harshitha   20     90   True           NaN   NaN   pass
2   mounisha   21     98   True           NaN   NaN   pass
3    punitha   20    106   True           NaN   NaN   pass
```

```
d.sort_values('marks',ascending=True)
```

	student	age	marks	grade	Grade_Letter	pass	result
1	harshitha	20	90	True	NaN	NaN	pass
2	mounisha	21	98	True	NaN	NaN	pass
3	punitha	20	106	True	NaN	NaN	pass
0	sravya	19	111	True	NaN	NaN	pass

```
d.sort_values('marks',ascending=False)
```

	student	age	marks	grade	Grade_Letter	pass	result
0	sravya	19	111	True	NaN	NaN	pass
3	punitha	20	106	True	NaN	NaN	pass
2	mounisha	21	98	True	NaN	NaN	pass
1	harshitha	20	90	True	NaN	NaN	pass

```
d.sort_values(['grade','marks'],ascending=[True,False])
```

	student	age	marks	grade	Grade_Letter	pass	result
0	sravya	19	111	True	NaN	NaN	pass
3	punitha	20	106	True	NaN	NaN	pass
2	mounisha	21	98	True	NaN	NaN	pass
1	harshitha	20	90	True	NaN	NaN	pass

```
d.sort_index(ascending=True)
```

	student	age	marks	grade	Grade_Letter	pass	result
0	sravya	19	111	True	NaN	NaN	pass
1	harshitha	20	90	True	NaN	NaN	pass
2	mounisha	21	98	True	NaN	NaN	pass
3	punitha	20	106	True	NaN	NaN	pass

```
d.nlargest(4,'marks')
```

	student	age	marks	grade	Grade_Letter	pass	result
0	sravya	19	111	True	NaN	NaN	pass
3	punitha	20	106	True	NaN	NaN	pass
2	mounisha	21	98	True	NaN	NaN	pass
1	harshitha	20	90	True	NaN	NaN	pass

```
d.nsmallest(3,'age')
```

	student	age	marks	grade	Grade_Letter	pass	result
0	sravya	19	111	True	NaN	NaN	pass
1	harshitha	20	90	True	NaN	NaN	pass
3	punitha	20	106	True	NaN	NaN	pass

```
data = {
    'Department': ['IT','HR','IT','HR','IT'],
    'Name': ['Alice','Bob','Charlie','Diana','Eve'],
    'Salary': [70000, 50000, 80000, 55000, 75000],
    'YearsExp': [3, 5, 7, 2, 4]
}
df = pd.DataFrame(data)
```

```
df.groupby('Department')['Salary'].min()
```

Department	Salary
HR	50000
IT	70000

dtype: int64

```
employees = pd.DataFrame({
    'emp_id': [1, 2, 3, 4],
    'name': ['Alice', 'Bob', 'Charlie', 'Diana'],
    'dept_id': [10, 20, 10, 30]
})

departments = pd.DataFrame({
    'dept_id': [10, 20],
    'dept_name': ['Engineering', 'HR']
})
```

```
pd.merge(employees,departments,on='dept_id')
```